

Index

S. No.	Name of the Experiment	Page No.	Date of Experiment	Date of Submission	Remarks
01.	Setting up and configuring a malware analysis lab.	01-03	09-01-26		
02.	To perform Basic static analysis of windows binaries on a linux platform.	05-06	16-01-26		
03.	To install flare floss and extract decoded and Obfuscated Strings.	07-09	30-01-26		
04.	Malware Detection using YARA Rules.	10-12	13-02-26		
05.	Understanding C code Constructs in Assembly language.	13-15	06-03-26		
06.	Analyze executable file using IDA Pro and underst and its Structure	16-18	27-03-26		

01. AIM: Setting up and Configuring a Malware Analysis lab.

02. OBJECTIVE:

- a. To safely Set up an isolated Environment for malware analysis
- b. To configure tools require for analyzing malicious file.
- c. To understand Secure handling and execution of malware Samples.

03. PROCEDURE:

Step 1: Instal Virtualization Software.

- a. Download and install VirtualBox or VMware.
- b. Enable window virtualization (VT-x/AMD-V) in BIOS.

Step 2: Create Virtual Machine

- a. create a new VM:
- b. RAM: 2-4 GB
- c. Disk: 40-60 GB
- d. Install windows 10 (recommended for malware Testing).

Step 3: Configure Network Isolation.

- a. Set network mode to Host-only Adapter.
- b. Disable:
 - Internet access
 - Shared clipboard access (optional but safety)

Step 4: Take snapshot

- a. after OS Setup:
 - Install basic tools
 - Take snapshot → "clean State".

Step 5: Install Analysis Tools

a. Install inside VM:

- Process Monitoring → Process Explorer
- File / registry monitoring → Procmon
- Network analysis → Wireshark
- Registry tracking → Regshot.

Step 6: Disable Security Features (Inside VM only)

a. Turn off:

- Windows Defender.
- Firewall.

(only inside isolated VM)

Step 7: Transfer Malware Sample / Create a Shared folder:

- a. configure a shared folder between the host and Kali VM
- b. Use the shared folder strictly for transferring analysis tools or malware sample.
- c. Disable the shared folder after file transfer to reduce risk.
- d. Avoid enabling clipboard sharing or drag-and-drop.

Step 8: Revert Snapshot.

- a. Shutdown the VM after configuration.
- b. Take a snapshot of the VM and name it "clean state"
- c. Confirm that the snapshot can be restored successfully.
- d. Use this snapshot as a baseline before executing any malware behaviour.

04. OBSERVATION:

- a. VM boot clearly with 2-4 GB RAM, network isolation.
- b. All tools are installed for analysis of malware.
- c. Shared folder mounts for file transfer.
- d. Snapshot taken to revert / restore VM to pre-analyze baseline within 30 seconds.

05. RESULT:

- a. Successfully created a secure malware analysis lab.
- b. Configured isolated Environment to prevent infection spread.
- c. Performed safe execution and monitoring of malware.
- d. Learned importance of snapshots & isolation.

01. AIM : To perform Basic Static Analysis of windows binaries on a Linux platform.

02. OBJECTIVES:

- a. To prepare a safe analysis environment in linux
- b. To examine the malware sample without executing it.
- c. To identify the file type and basic characteristics of the sample.
- d. To generate hash values for integrity, checking, and threat intelligence lookup.
- e. To extract readable strings, and identify suspicious indicators such as URLs, IP addresses, commands, registry paths, and API names.
- f. To check whether the file is packed or obfuscated.

03. PROCEDURE:

01. Prepare the Analysis Environment:

- a. Boot into a Linux system (kali)
- b. Ensure internet connectivity is disabled.
- c. log in using non-root user account.

02. create a dedicated malware analysis directory:

- a. create a folder for malware samples using below command:

`mkdir malware_analysis.`

03. Copy Malware Samples:

- a. Transfer windows binary via shared folder and verify using below command:

`ls -l`

Step 4: Identify File type:

a. confirm that the file type is a windows executable using below command:

file malware.exe

Step 5: Generate hashes:-

a. Generate cryptographic hashes using below command to Verify on Virustotal:

md5sum malware.exe ➡ hashes.txt

Sha5Sum malware.exe >> hashes.txt

Sha25Sum malware.exe >> hashes.txt

Step 6: Extract printable Strings (Ascii):

Strings malware.exe > strings.txt.

Step 7: Extract stack and decoded Strings using floss.

floss malware.exe > output.txt

Step 8: View extracted file-

To View extracted strings, use below Command:

cat output.txt or nano strings.txt

Step 9: Search for Suspicious indicators:

(a) Search for URL: `grep -i "http" strings.txt`

(b) Search for IP address: `grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}' strings.txt`

(c) Search for command execution: `grep -i "cmd" strings.txt`

(d) Search for registry ^{Entries}: `grep -i "Software\Microsoft" strings.txt`

(e) Search for Suspicious windows APIs.

`grep -iE "createProcess|winExec|RegSetValue|writeFile" strings.txt.`

Step 10: Identify Possible Obfuscation or Packing

a. To identify packers use detect-it easy

b. Another option is CLI command "pepack"

pepack malware.exe.

4. OBSERVATIONS:

a. File confirmed PE32 Windows executable file.

b. Hash values were successfully generated and saved.

c. Readable strings were extracted from the file using static analysis tools.

d. Some strings indicated possible indicators of compromise, including URLs, registry-related content, and windows API reference.

e. Packing and obfuscation checks helped determine whether the malware used concealment techniques.

5. RESULT:

a. Basic static analysis of the given malware sample was completed successfully.

b. The file type was confirmed, hash values were generated, suspicious strings were extracted, and indicators such as URLs, persistence-related entries, and API calls were identified.

c. The sample was also checked for packing or obfuscation, making the analysis useful for further malware investigation.

01. AIM: To Install Flare-floss in a Seperate Virtual environment on kali linux and extract decoded and obfuscated Strings from a windows PE file for malware analysis.

02. OBJECTIVES:

(a) To create an isolated Python Virtual environment for FLARE-FLOSS.

(b) To install and configure Flare-Floss in kali linux.

(c) To analyze a window PE File using Flare-Floss.

(d) To extract Static, decoded and deobfuscated strings from executables.

(e) To identify Suspicious indicators such as URL, file path, command and API names.

03. PROCEDURE:

(i) Create Seperate Virtual environment using:

```
mkdir ~/floss-env
```

```
cd ~/floss-env
```

```
python3 -m venv floss_env
```

(ii) Activate the environment using:

```
source floss-env/bin/activate
```

(iii) Installing floss using:

```
pip install flare-floss
```

(iii) Extract string using FLARE-FLOSS:

a. Basic String Extraction:

```
floss <malware.exe>
```

It extracts static string, stack string, Decoded string.

b. Save output to file:

```
floss <malware.exe> > floss_output.txt
```

(iv) Search for Suspicious Indicators:

a. Search for extracted strings:

```
cat floss_output.txt | grep -i http
```

```
cat floss_output.txt | grep -i cmd
```

```
cat floss_output.txt | grep -i powershell
```

b. Look for:

a. URLs / IP Addresses

b. File paths.

c. Registry keys.

d. Suspicious commands.

e. API calls.

DA- OBSERVATION:

a. A separate python environment was successfully created for FLARE-FLOSS

b. FLARE-FLOSS was installed and executed without affecting system python

c. Static strings such as file paths & APIs were extracted.

d. Decoded strings revealed hidden or obfuscated strings.

e. Suspicious indicators such as URL, command-line instruction & system related strings were identified.

f. Output was successfully saved for further malware analysis.

05. RESULT:

- a. FLARE - FLOSS was Successfully installed and used in an isolated environment on kali Linux
- b. The tool efficiently extracted static & decoded strings from the windows PE file, helping identify potentially malicious indicators & improving the overall static malware analysis process.

01. AIM: To create and apply YARA Rules for identifying malware based on patterns and signatures.

02. OBJECTIVES:

- a. To understand the Purpose of YARA in malware detection.
- b. To install and use YARA in a windows environment.
- c. To create a simple YARA rule File.
- d. To scan a test file using the created rule.
- e. To observe how YARA identifies suspicious content based on matching Strings.

03. PROCEDURE:

Step 1: Installation of YARA.

a. Download YARA

- Visit Official Github page of YARA
- Download latest release for your OS.

b. Install YARA

- Extract Downloaded files.
- Add YARA folder to system PATH (for command-line use)

c. Verify Installation:

- Open command Prompt.
- Type:
Yara --version.
- Confirm installation is successful.

Step 2: Writing Basic YARA Rules.

a. Create Rule File:

- Open text editor
- Save file as sample_rule.yar

b. Write Basic YARA Rule:

rule Suspicious_string

{

Strings:

\$a = "Malware"

\$b = "hack"

\$c = "cmd.exe"

Condition:

any of them

}

c. Save the file.

Step 3: Applying YARA Rule

a. Run YARA Scan:

- Open command prompt
- Execute:

YaraB4.exe rule.Yar Suspicious_file.exe

b. Analyze Output:

- If matched → rule name will be displayed.
- If no match → no output.

Step 4: Advanced Rule (optional)

a. Add File Size Condition.

- condition:

fileSize < 1 MB and any of them.

b. Use Multiple Conditions:

- Combine Strings and logic for accuracy.

04. OBSERVATION:

- a. YARA executed Successfully from the command line.
- b. The rule file "Sample rule.yar" was created without errors.
- c. The test file contained the word "malware".
- d. When the scan command was run, YARA matched the string in the file.
- e. The output displayed the rule name, and the file name, showing that the rule worked correctly.
- f. This experiment showed that YARA can detect suspicious files using custom string-based rule.

05. RESULT:

The Practical was completed Successfully. A YARA Rule was created and applied to a test file. The tool correctly detected the Suspicious String present in the file, demonstrating how YARA can be used for malware detection based on patterns & Signatures.

01. AIM: To understand how high-level C language construct are translated into Assembly language.

02. OBJECTIVES:

- a. To write a Simple C program.
- b. To convert the C program into Assembly code using GCC.
- c. To Study how Variables, Arithmetic operations, and function calls appear in Assembly.
- d. To understand basic Stack and register usage during program execution.
- e. To relate high-level program logic with low-level machine instructions.

03. PROCEDURE:

Step 1: Writing C Program.

a. Create a C Program.

01. `#include <stdio.h>`

02.

03. `int main()`

04. `{`

05. `int a = 5, b = 10, c;`

06. `c = a + b;`

07. `printf("%d", c);`

08. `return 0;`

09. `}`

b. Save File

• Save as program.c

Step 2: Convert C to Assembly

a. Compile to Assembly

- Open terminal
- Run:

```
gcc -S program.c
```

b. View Assembly File

- Open generated file program.c

Step 3: Analyze Assembly code

a. Identify Sections:

- .data → Variables.
- .text → instructions.

b. Understand Variable Storage:

- Observe how a, b, c are stored.

c. Analyze Instructions:

- Example:

```
movl $5, -4(%rbp)
```

```
movl $10, -8(%rbp)
```

d. Arithmetic Operation:

- Identify addition:

```
addl %eax, %edx
```

e. Function call

- Observe printf call in assembly.

Step 4: Stack and Registers.

a. Understand Stack Frame:

- rbp → base pointer.
- rsp → stack pointer.

b. Trace Execution:

- Follow instruction flow step-by-step.

04. OBSERVATION:

- The C program was successfully converted into Assembly code.
- Variable assignments in C were translated into mov instructions in Assembly.
- The addition operation was represented using Arithmetic instructions such as addl.
- The output function printf() appeared as a function call in Assembly.
- The stack frame showed how local variables were managed using registers like rbp and rsp.
- The practical helped in understanding the relationship between high-level C statements and low-level instructions.

05. RESULT:

The practical was completed successfully. The generated Assembly code clearly showed how C language constructs such as variable declaration, assignment, arithmetic operation, and function call are translated into low-level instructions. This improved understanding of program execution at machine level.

01. To Analyze an executable file using IDA Pro and understand its structure and behavior through disassembly.

02. OBJECTIVES:

- (a) To perform Static analysis of an executable file.
- (b) To identify entry point and functions.
- (c) To analyze Strings and API calls.
- (d) To understand basic control flow.
- (e) To observe program behavior.

03. PROCEDURE:

Step 1: IDA PRO (IDA Free) was installed & launched on the System.

Step 2: Auto Analysis

- (a) IDA Automatically performs disassembly.
- (b) It identifies:
 - Functions
 - Segments
 - Imported libraries.

Step 3: Entry Point Analysis.

- (a) Navigate to entry point (e.g. $0x401000$)
- (b) Observe initial instructions.
- (c) Trace execution flow towards main function.

Step 4: String Analysis.

- (a) Open strings window (Shift + F12)
- (b) Identify readable Strings.

(c) look for suspicious strings:

- URL
- cmd
- Registry paths.

Step 5: Cross-Reference Analysis

- (a) Press x on selected strings / functions
- (b) Analyze where & how they are used in the program.
- (c) link suspicious strings to their respective functions.

Step 6: Control Flow Analysis.

- (a) Switch to Graph View (spacebar)
- (b) Analyze: Branches, loops, Execution paths.
- (c) Identify repeated patterns (e.g. network retry loops)

Step 7: API call Analysis

- (a) Rename unknown functions (Sub_XXXXX) based on behavior
- (b) Use meaningful names like:
 - network_connect.
 - execute_command.
 - registry_persistence.

Step 8: Behavioral Analysis

- (a) combine findings from:
 - Strings
 - API calls
 - control flow.
- (b) Identify malware capabilities:

- File creation.
- Persistence.
- Command execution.
- Network communication.

04. OBSERVATION:

- (a) The executable file was successfully loaded & analyzed.
- (b) Functions, Segments, & imports were identified automatically.
- (c) Entry point showed normal program initialization.
- (d) Strings included system paths & DLL References, no suspicious data found.
- (e) API calls were standard windows functions.
- (f) Control flow showed normal execution without anomalies.
- (g) No indicators of malicious activity were observed.

05. RESULT:

The executable was successfully analyzed using IDA Free. The analysis revealed normal program behavior with no signs of malicious activity. The practical demonstrated the use of IDA Pro for disassembly, string analysis, API identification, & control flow understanding.